

# Relatório Individual de Produção

## Projeto Pokédex — Etapa 1

Aluno: Luís Leão

Disciplina / Professor: Akira\_\_\_\_\_

Repositório do grupo (GitHub): <https://github.com/poo-ec-2026-1/g10>

Data: junho de 2026

## 1. Atribuição de cargo e tarefas

### Cargo / responsabilidade atribuída:

Fiquei responsável pela camada de Banco de Dados do projeto. Cabia a mim, a partir do arquivo CSV com os 151 Pokémon da primeira geração, estruturar a base de dados que servirá de fonte para as demais partes do sistema (interface e modelagem), bem como implementar o código de acesso a esses dados.

### Tarefas atribuídas a priori:

- Tratar o arquivo CSV de origem (padronização de cabeçalho, codificação).
- Implementar a conexão com o banco (classe Database).
- Implementar os repositórios (padrão Repository) que dão acesso aos dados.
- Implementar a rotina de carga (PopularBanco) que lê os CSVs e popula o banco.
- Documentar a camada (LEIA-ME e relatório técnico).

### O que foi exercido na prática:

Exerci as responsabilidades atribuídas e adotei o padrão recomendado pelo tutorial da disciplina: ORMLite sobre SQLite, com classe Database, classes Repository e uma classe de carga. Como o foco da minha parte são os dados e o acesso a eles, alinhei com a equipe que as classes de modelo (entidades) ficam sob responsabilidade da etapa de Modelagem; os meus repositórios operam sobre essas classes.

## 2. Contribuição de acordo com a atribuição

### O que foi cumprido:

- Tratamento do CSV original: cabeçalho padronizado (sem espaços e acentos), leitura em UTF-8 (preservando caracteres especiais como Nidoran♀ e Nidoran♂) e Tipo 2 ausente tratado como nulo.
- Implementação da classe Database (abertura/fechamento da conexão com o SQLite).
- Implementação dos repositórios PokemonRepository, GolpeRepository, NatureRepository e TipoEfetividadeRepository, com create, loadFromId, loadAll e consultas específicas (busca por nome, filtro por tipo e cálculo de fraquezas/vantagens).
- Implementação da classe PopularBanco, que lê os três CSVs (pokemons.csv, golpes.csv, pokemon\_golpes.csv) e popula o banco via ORMLite.
- Complemento de dados que não constavam no CSV: associação de um golpe característico a cada Pokémon (uso visual no card), inclusão da tabela canônica de efetividade entre os 18 tipos e das 25 naturezas.
- Validação dos dados gerados: 151 Pokémon, 91 golpes, 25 naturezas, 120 confrontos na tabela de efetividade; verificação de integridade referencial sem violações.

- Documentação: arquivo LEIA-ME.md com instruções de instalação das bibliotecas e execução, e Relatório Técnico em PDF descrevendo a camada de banco de dados.

#### **Três commits mais relevantes:**

- 1) <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — Estrutura da camada de banco: classe de conexão Database.java e repositórios (PokemonRepository.java, GolpeRepository.java, NatureRepository.java, TipoEfetividadeRepository.java).
- 2) <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — Rotina de carga PopularBanco.java e dados-fonte (pokemons.csv, golpes.csv, pokemon\_golpes.csv), incluindo a inserção dos dados canônicos (tabela de efetividade e natures).
- 3) <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — Documentação: LEIA-ME.md (instruções de instalação e execução) e Relatorio POO.pdf (relatório técnico da camada).

#### **O que não deu para cumprir / em aberto:**

- Persistência do time montado pelo usuário: ainda depende de decisão da equipe (salvar no banco ou manter em memória).
- Edição de golpes adicionais por Pokémon: foi definido um golpe característico (uso visual); ampliar para mais de um depende da definição da equipe sobre a tela de time.

#### **Principais dificuldades:**

- Aprender o padrão ORM (ORMLite) a partir do tutorial da disciplina e aplicá-lo a uma base maior que a do exemplo.
- Configurar as cinco bibliotecas (sqlite-jdbc, ormlite-core, ormlite-jdbc, slf4j-api e slf4j-simple) no BlueJ.
- Decidir como representar dados que não estavam no CSV (tabela de efetividade e natures) sem comprometer a simplicidade da camada.

### **3. Contribuição além do atribuído**

Além do que foi atribuído estritamente à camada de banco, contribuí com a equipe nos seguintes pontos:

- Tratei o CSV original e o disponibilizei em formato limpo (UTF-8, cabeçalho padronizado) para uso compartilhado por toda a equipe.
- Produzi o arquivo LEIA-ME.md do banco, descrevendo as bibliotecas necessárias e o passo a passo no BlueJ — material reaproveitável por quem for executar o projeto.
- Elaborei um relatório técnico em PDF descrevendo a camada de banco de dados (arquitetura, decisões e resultados), que serve como documentação do projeto no repositório do grupo.
- Propus a arquitetura em camadas (Conexão → Repositórios → Modelo) adotada pelo grupo: definimos que as classes de modelo (entidades) ficam sob responsabilidade da etapa de Modelagem e que minha camada opera sobre elas, o que reduziu retrabalho e alinhou as fronteiras entre as partes do projeto.
- Ajudei colegas a entender como a interface deve consumir os repositórios (sem escrever SQL), apresentando exemplos de chamadas (loadAll, loadByName, loadByType e cálculo de fraquezas) que servem de base para a Tela 1 (Pokédex), o card ampliado e o painel de análise.

#### **Commits / documentos adicionais relevantes:**

- <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — CSV tratado disponibilizado no repositório (pokemons.csv com cabeçalho padronizado e codificação UTF-8).
- <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — LEIA-ME.md do banco (instruções de instalação das bibliotecas e execução no BlueJ).
- <https://github.com/poo-ec-2026-1/g10/commit/3247b34773df46867870f4544bef306dfb6de132> — Relatorio POO.pdf (relatório técnico da camada de banco de dados).

## 4. Considerações gerais

### O que aprendi:

- Aplicação prática do padrão ORM (ORMLite) e do padrão Repository: substituir SQL escrito à mão por operações sobre objetos Java reduz a complexidade do código e melhora a separação de responsabilidades.
- Importância de tratar a fonte de dados (CSV) antes da carga: pequenas decisões como codificação UTF-8 e tratamento de campos vazios evitam problemas que só apareceriam mais tarde, na interface.
- Como a camada de dados se conecta ao restante do projeto sem acoplamento: a interface não escreve SQL e não conhece o banco; consome apenas objetos retornados pelos repositórios.

### Trabalhos futuros / pendências:

- Definir, em conjunto com a equipe, se o time montado pelo usuário será persistido no banco (já existe estrutura prevista) ou mantido apenas em memória durante a execução.
- Caso a equipe decida ampliar o número de golpes por Pokémon ou incluir descrição textual no card, estender o esquema e os repositórios de forma incremental.
- Acompanhar a integração com a Modelagem (UML) e a Interface (JavaFX), garantindo que os nomes de atributos e métodos permaneçam compatíveis com os repositórios desta camada.

### Conclusão:

A camada de banco de dados foi entregue concluída e validada, dentro do padrão técnico exigido pela disciplina (ORM + Repository). A contribuição vai além do escopo estrito do componente, abrangendo tratamento de dados, documentação do projeto e proposta da arquitetura adotada pelo grupo, o que permitiu que as demais etapas (Modelagem e Interface) avançassem sobre uma base consistente.